

Care Plan Generation Module — Design

Module: `hdh.modules.careplan` · **Status:** design (not yet built) · **Date:** 2026-08-06

Contributors

Name	Role	Contribution
Ajmal Mahmood	Author / Architect	Original design, clinical framework, reference analysis

To be added as a contributor: submit design feedback or implementation work via a PR or issue on this document, and add yourself to the table in the same change.

A design for an AI-assisted care-plan generation module: a **subagent** the existing agent pipeline can delegate to, which turns a structured patient record into an HL7/FHIR-shaped care plan — concerns, goals, interventions, and outcomes — grounded in curated knowledge bases via RAG, auto-evaluated against expert rubrics, and finalized through a checkpointed human-in-the-loop approval workflow.

This document is the specification we will build against. It follows the structural and generation framework of the accompanying reference analysis ("*What a care plan actually is, structurally*" — included alongside this design), applied to hdh's architecture and educational mission.

1. Purpose and positioning

hdh already demonstrates a validated agentic pipeline over synthetic EHR data. Care-plan generation is the natural capstone: it is the hardest kind of clinical artifact to generate well — multi-section, cross-referenced, guideline-bound, safety-critical, and inherently collaborative. It exercises every architectural muscle the project teaches:

- **Subagent composition** — a LangGraph subgraph invoked as a tool by the main pipeline's tool executor (or standalone via CLI)
- **Schema registry** — the plan's entities enter the data model as a schema module (new tables, declaratively)
- **RAG over curated knowledge** — retrieval tools per plan section, backed by pluggable stores
- **Constrained generation** — the LLM selects and adapts from vetted, retrieved candidates; it never free-generates clinical recommendations
- **Rubric-driven auto-evaluation** — an evaluation knowledge base as the quality gate before any human sees the draft
- **Checkpointed human-in-the-loop** — the workflow pauses for approval and resumes when the human responds, however much later

Non-goals. This is an educational demonstration over synthetic data. It is not a medical device, not clinical decision support for real patients, and it does not aim for full MCC eCare Plan IG profile conformance in v1 — it aims to *teach the shape* of a conformant system.

2. Grounding: what we are generating

2.1 The four-part graph

Every serious care-plan standard reduces to the same structure, inherited from the nursing process (ADPIE) and codified in C-CDA R2.1:

Health Concerns → Goals → Interventions → Outcomes / Evaluations

with **every edge explicitly traced**. A goal that addresses no concern is noise; an intervention that serves no goal is padding. Traceability is the single most useful validation invariant available, and it is enforced deterministically in this design (§8) — not left to the LLM.

2.2 FHIR mapping

CarePlan is a thin spine; the content lives in the resource graph it points into. The module's internal entities (§5) map onto it for export:

Plan element	FHIR resource	Notes
Concerns	Condition (category health-concern)	SNOMED CT / ICD-10-CM — our ontology module already dual-codes
Goals	Goal	target.measure (LOINC) + detailQuantity + due ; achievementStatus ; expressedBy distinguishes patient-stated from clinician-stated
Interventions	ServiceRequest , MedicationRequest , NutritionOrder , CommunicationRequest	referenced from CarePlan.activity
Assignments	Task	owner may be a RelatedPerson (caregiver)
Team	CareTeam + RelatedPerson	family members are first-class
Outcomes	Observation , QuestionnaireResponse	PROs, device data
Authorship	Provenance	per-element; carries AI-vs-human attribution (§10)

2.3 The implementation-guide stack (knowledge sources)

Source	Role in this module
MCC eCare Plan IG (STU1)	The primary template: multimorbidity, outpatient, longitudinal, patient-and-caregiver-inclusive. Its hand-authored example instances are our gold reference for what a good structured plan looks like.
Gravity Project SDOH Clinical Care IG	Social risks as coded concerns + closed-loop referral interventions — required for cases like "lives alone"
Personal Functioning and Engagement IG (ex-PACIO)	ADLs, functional/cognitive status — feeds feasibility assessment
Medication-safety criteria (Beers/STOPP-START-derived)	Risk flags such as <i>sulfonylureas in the elderly</i> — the hypoglycemia example in §12
Condition guidelines (e.g., ADA Standards older-adult section)	Deintensification targets, monitoring intervals
Evaluation rubrics (expert-curated, ours)	The auto-evaluation KB (§9) — modeled on HealthBench-style rubric grading

Licensing note: HL7 IG content is openly licensed and ingestible. AGS Beers Criteria and published nursing care-plan handbooks are copyrighted — we ingest only openly licensed derived summaries we author ourselves, and the corpus manifest records provenance and license per document.

3. Requirements

Functional

1. Input: a structured patient record (the hdh chart — demographics, problem list with control status, visits, medications, labs, vitals) identified by MRN.
2. Output: a care plan as (a) rows in new registry-defined tables, (b) a FHIR R4 `CarePlan` bundle export, (c) a human-readable rendering.
3. Section generation via **section-specific retrieval tools** over the knowledge bases: concerns (with risk statements), goals, interventions, outcome measures, care-team suggestions.
4. AI-assisted, not AI-authored: the user can add/edit/remove any element; the plan records which elements are AI-proposed vs human-entered.
5. Auto-evaluation stage scoring the draft against curated rubrics before human review.
6. Human-in-the-loop approval with **checkpointing**: the workflow pauses at review, persists all state, and resumes on the human's response — minutes or days later, across process restarts.
7. Invocable two ways: as a **tool of the main agent pipeline** ("generate a care plan for MRN...") and standalone via `hdh careplan ... CLI`.

Non-functional

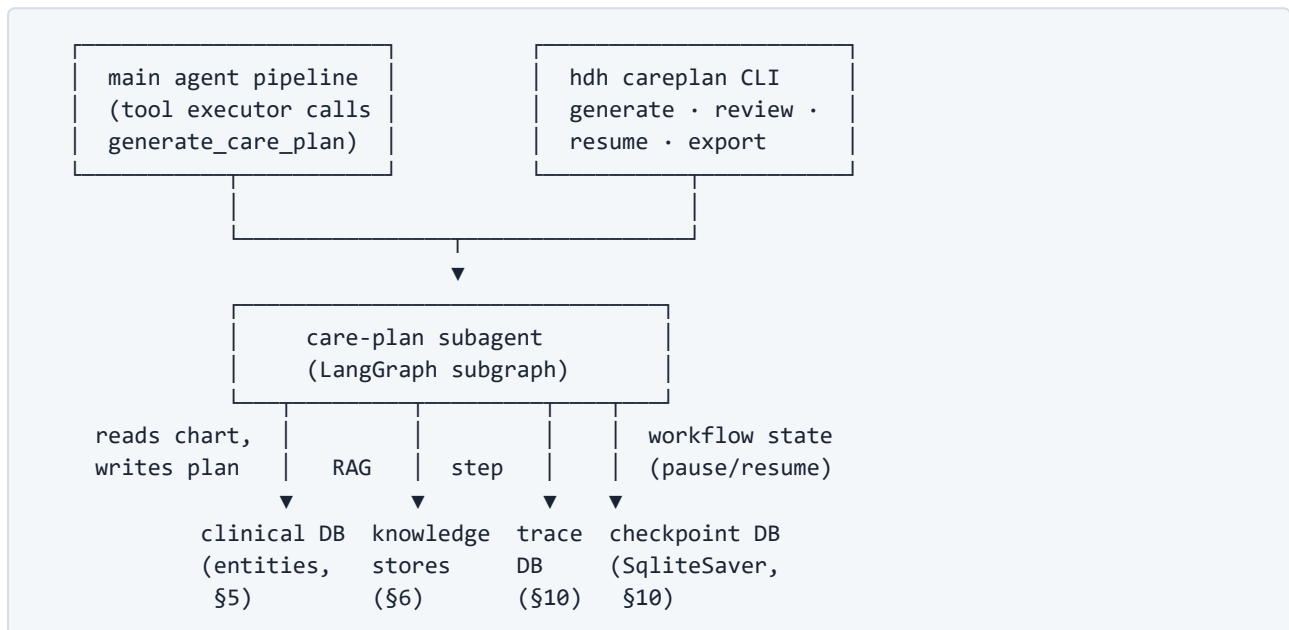
- All hdh gates apply: contracts, DI, pluggability, immutability where possible, injection safety, tests without an API key (fakes at every seam), full tracing of every LLM/tool step with token accounting.

- Deterministic wherever determinism is possible (candidate retrieval, traceability validation, safety flags); LLM only where judgment is the task (selection, adaptation, phrasing, evaluation narrative).
- Cost-bounded: per-section retrieval keeps context small (the token-economy lessons from the pipeline apply here).

4. Architecture overview

Two views: where the subagent sits (context), and what it does (workflow).

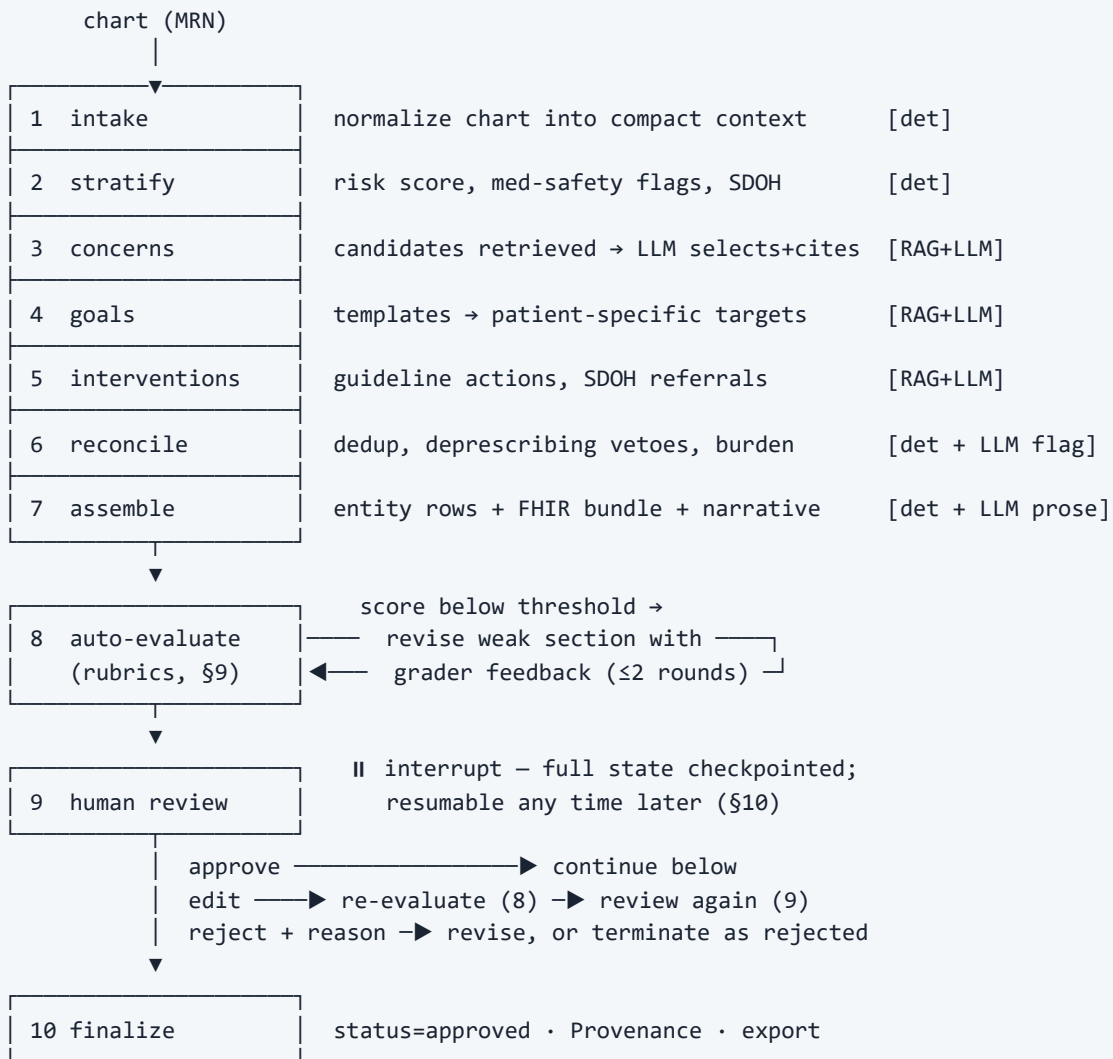
4.1 System context — who talks to what



One subagent, four storage concerns, each with a single responsibility: clinical content, retrievable knowledge, observability, and resumable workflow state.

4.2 Workflow — the stages in order

A single forward spine; the only two loops are drawn where they occur. Each stage is labeled with its mechanism — **[det]** = deterministic code, **[RAG+LLM]** = retrieval then constrained selection (§7).



Reading it as a sentence: *deterministic preparation (1–2), constrained generation (3–5), safety reconciliation (6), assembly (7), machine grading (8), human decision (9), finalization (10)* — with revision loops only out of grading and review, never inside generation.

Design rules carried over from the pipeline: every node's dependencies are injected (a `CarePlanDeps` frozen dataclass mirroring `PipelineDeps`), every node execution is recorded as a trace step with tokens and duration, and the whole graph runs offline in tests with fake retrieval and fake LLMs.

5. Data model — a schema-registry module

`careplan` ships as a **schema module** (manifest + `schema/entities/*.json` + `schema/relationships/*.json`), exercising the registry's new-entity path:

Entity	Key columns	Traces to
CarePlanRecord	id, patient_id→patients.id, status, title, created_at, updated_at, checkpoint_thread_id, narrative	—
HealthConcern	id, care_plan_id, concern_type (condition/risk/sdoh/functional), code (ICD-10/SNOMED/Z-code), statement, priority, source (ai / human), evidence_refs (JSON)	chart data
PlanGoal	id, care_plan_id, concern_id→health_concerns.id, statement, measure_loinc, target_value, target_due, expressed_by (patient/clinician), status, source	concern
PlanIntervention	id, care_plan_id, goal_id→plan_goals.id, intervention_type (medication/service/referral/education/monitoring), statement, code, owner_role, schedule, source	goal
PlanOutcome	id, care_plan_id, goal_id, measure, observed_value, observed_at, achievement_status	goal
PlanEvaluation	id, care_plan_id, rubric_id, dimension_scores (JSON), overall, verdict, narrative, evaluated_at	plan

Notes:

- The **foreign-key chain enforces the four-part graph**: every goal points at a concern, every intervention at a goal, every outcome at a goal. Orphans are structurally impossible to persist — the deterministic realization of the traceability invariant.
- `source` on every content row is the AI-vs-human provenance bit; the FHIR export writes it into `Provenance` and `Goal.expressedBy`.
- `status` lifecycle on `CarePlanRecord`: `draft` → `ai_generated` → `auto_evaluated` → `pending_review` → `user_edited` → `approved` | `rejected` (rejection carries a reason and may loop to revise).
- `checkpoint_thread_id` binds the row to the LangGraph checkpoint (\$10).

6. Knowledge layer

6.1 Store abstraction (pluggable, like GapFinder)

```
class KnowledgeStore(Protocol):
    name: ClassVar[str]
    def search(self, query: str, corpus: str, k: int = 5,
               filters: Mapping | None = None) -> list[KnowledgeHit]
    def ingest(self, corpus: str, documents: Iterable[KnowledgeDoc]) -> int

@dataclass(frozen=True)
class KnowledgeHit:
    corpus: str; doc_id: str; chunk: str; score: float
    source: str; license: str; metadata: Mapping
```

Two shipped implementations:

Store	Backend	Why
Fts5Store (default)	SQLite FTS5 (BM25) in ~/.hdh/knowledge.db	Zero dependencies, fully offline, deterministic — tests and the practitioner path run on this
VectorStore (optional extra)	sqlite-vec (or Chroma) + an embedding provider behind an injected embed() callable	Semantic retrieval where lexical match fails

Production-style deployments would use hybrid retrieval (BM25 U vector, reciprocal-rank fusion) — the protocol supports it as a third composite implementation without touching callers. Retrieval hits always carry `source` and `license`, so **every AI-proposed plan element can cite the knowledge chunk(s) it came from** (`evidence_refs`) — the audit-trail property that rules out baking this knowledge into fine-tuned weights.

6.2 Corpora

Each corpus is a directory of Markdown/JSON documents with a `corpus.json` manifest (name, version, source, license, ingest date):

```
knowledge/
├─ mcc_ecare_plan/      # IG narrative, profile summaries, example plan instances
├─ gravity_sdoh/        # SDOH concern codes (Z55-Z65), referral interventions
├─ personal_functioning/ # ADL/IADL, functional & cognitive status concepts
├─ med_safety/          # self-authored Beers/STOPP-START-derived risk statements
├─ condition_guidelines/ # per-condition monitoring/goal templates (ADA older-adult, ...)
└─ eval_rubrics/        # §9 – expert-curated evaluation templates & rubrics
```

`hdh careplan ingest [--corpus X]` (re)builds the store; the manifest hash makes ingestion idempotent. Corpora live in the repo (they are small, curated text — not the copyrighted originals).

6.3 Section-specific retrieval tools

The subagent does not get one generic "search" tool; it gets **one tool per plan section**, each pre-scoped to the right corpora and filters — the same selective-context lesson the pipeline's token economy taught:

Tool	Corpora	Example query it answers
<code>retrieve_concern_knowledge(conditions, meds, age, social)</code>	med_safety, gravity_sdo, condition_guidelines	"elderly + glipizide + lives alone" → hypoglycemia-risk statement, social-isolation Z-code
<code>retrieve_goal_templates(concern)</code>	condition_guidelines, mcc_ecare_plan	A1c target template with older-adult deintensification
<code>retrieve_interventions(goal, patient_context)</code>	condition_guidelines, gravity_sdo, personal_functioning	CGM w/ caregiver alerts; Meals-on-Wheels closed-loop referral
<code>retrieve_outcome_measures(goal)</code>	mcc_ecare_plan, condition_guidelines	LOINC-coded measures + review intervals
<code>retrieve_rubric(plan_type)</code>	eval_rubrics	the grading template for \$9

7. Generation pipeline (nodes)

The ordering follows the reference analysis: **deterministic first, LLM downstream and constrained.**

1. **intake** — load the chart (reusing `patient_to_json`), normalize the problem list and med list into a compact structured context.
2. **stratify** — deterministic flags, no LLM: reuse the risk module's score; age/med-class rules from `med_safety` (sulfonylurea + age ≥ 65 → hypoglycemia flag); polypharmacy count; SDOH markers present in the record. Output: a flag set with sources.
3. **concerns** — for each condition/flag, retrieve candidate concern statements; the LLM *selects, prioritizes, and phrases* concerns from the candidates (schema-enforced JSON), each carrying `evidence_refs`. It may not emit a concern with no retrieval support and no chart support.
4. **goals** — per concern, retrieve goal templates; LLM instantiates them with patient-specific targets (again: selection + parameter filling, not invention). Patient-expressed goals (entered later by the user) outrank guideline defaults on conflict.
5. **interventions** — per goal, retrieve intervention candidates; LLM selects and schedules; each intervention names an owner role.
6. **reconcile** — deterministic + LLM hybrid, the step where naive systems fail: de-duplicate interventions across conditions, apply med-safety vetoes (the deprescribing layer — e.g., *remove* rather than add for a flagged sulfonylurea), and compute a simple treatment-burden score (intervention count, visit frequency, device count) with an LLM pass asked only to *flag* likely-excessive burden for human attention.
7. **assemble** — write entity rows (`status=ai_generated`), build the FHIR bundle, draft the narrative (`CarePlan.text`) — the one place free-form LLM prose is welcome, because it renders structure that already exists.

Structural validation runs after assemble, deterministically: complete edge coverage (guaranteed by FK schema, asserted anyway), every AI element has non-empty `evidence_refs`, codes exist in our terminology tables, and med-safety vetoes were honored.

8. Why the graph invariant is not an LLM job

The four-part traceability rule is checked three ways, none of them by generation-time prompting alone: (1) the schema makes orphan rows unrepresentable; (2) assemble-time assertions verify coverage in both directions (every concern has ≥ 1 goal or an explicit "monitor-only" marker; every goal ≥ 1 intervention; every goal an outcome measure); (3) the auto-evaluator scores coherence *quality* (are the edges clinically sensible, not merely present). Cheap, deterministic checks first; judgment where only judgment works.

9. Auto-evaluation

A distinct pipeline state, mirroring the validator stage of the main pipeline but rubric-driven:

- **Rubric KB** (`eval_rubrics` corpus): expert-curated templates per plan archetype (multimorbid elderly, single-chronic-condition adult, ...), each a set of dimensions with anchored score descriptions — modeled on HealthBench-style behavior-level rubrics. Dimensions for v1: *completeness* (all significant chart problems addressed), *traceability quality*, *guideline concordance*, *safety* (flags addressed, no contraindicated proposals), *goal quality* (SMART, patient-centered), *feasibility/burden*, *readability of narrative*.
- **Grader**: one schema-enforced LLM call per dimension group, given the plan + the rubric + the chart summary; emits per-dimension scores with cited justification. Deterministic checks (§7) are injected as pre-computed facts so the grader doesn't re-derive them.
- **Outcome**: scores below threshold route back to the relevant section node with the grader's reasons as feedback — the same bounded retry-with-feedback loop the main pipeline uses (max 2 revision rounds), then proceed to human review regardless, with low scores prominently displayed. Auto-evaluation *informs* the human; it never auto-approves.

10. Human-in-the-loop with checkpointing

- The subgraph is compiled with a **LangGraph checkpoint** (SqliteSaver in `~/hdh/careplan_checkpoints.db`); the review node issues an `interrupt()`. Each plan's `checkpoint_thread_id` names its thread.
- `hdh careplan review <plan-id>` renders the draft (sections, sources, rubric scores), then captures the human action:
- **approve** → resume → finalize (status `approved`, Provenance written)
- **edit** → structured edits (add/modify/remove concern/goal/intervention, entered as `source="human"`) → resume → re-run *only* auto-evaluation on the edited plan → back to review
- **reject** --reason → resume → revise loop with the reason as feedback, or terminate as `rejected`
- Because state lives in the checkpointer *and* the entity rows, the process can die and resume days later: `hdh careplan resume <plan-id>` re-attaches to the thread. This is the same durability idea as the pipeline's run/turn/step tracing, applied to workflow state.
- Every generation and review step is also recorded in the **existing trace DB** (a care-plan run is a `run` with turns per node), so cost and behavior are inspectable with `hdh trace` like everything else.

11. Integration surface

Surface	Contract
Agent pipeline tool	<code>generate_care_plan(mrn) -> plan_id + summary</code> — registered into the executor's toolset (intent <code>care_plan</code> added to <code>INTENT_TOOLS</code>); the subagent runs synchronously up to its interrupt, returns "draft ready for review: plan 17"
CLI	<code>hdh careplan generate --mrn / show / review / resume / export --format fhir / ingest</code>
FHIR API	<code>GET /CarePlan?patient=<mrn></code> serving the export bundle (extends the <code>fhir_api</code> module)
Extras	<code>careplan = [langgraph, ...]</code> reusing the agent extra's stack; vector store optional

12. Worked example (design fixture)

82-year-old, T2DM (uncontrolled) + CKD, on glipizide, lives alone — the scenario every layer must handle end-to-end; it becomes the canonical test fixture and demo:

1. **stratify**: risk-module score high; `med_safety` rule fires — sulfonylurea + age $\geq 65 \rightarrow$ *hypoglycemia risk*; lives-alone marker $\rightarrow$ SDOH.
2. **concerns** (retrieved + selected): uncontrolled T2DM with CKD; **risk of severe hypoglycemia** (glipizide in an elderly patient living alone — delayed rescue), citing the med-safety chunk; social isolation (Z60.2); fall risk.
3. **goals**: *zero severe hypoglycemic events* (patient-safety goal, deintensified A1c target $\sim 7.5\text{--}8.0\%$ per older-adult guidance — not the younger-adult 7.0%); *maintain independent ADLs*.
4. **interventions**: deprescribing review of glipizide (safer agent per renal function); glucose monitoring with caregiver-shared alerts; Meals-on-Wheels referral (Gravity closed-loop pattern); fall assessment; medication blister packaging. Each traced to its goal, each citing its source.
5. **reconcile**: burden check — flags device + 3 referrals for review.
6. **auto-evaluate**: safety dimension verifies the hypoglycemia flag was *addressed by* an intervention (not merely listed).
7. **review**: clinician adds a patient-expressed goal ("keep gardening"), marks approve $\rightarrow$ finalized, exported as FHIR with dual-coded concerns.

13. Testing & evaluation strategy

- **Unit/offline**: every node with injected fakes (fake retrieval hits, fake LLM callables) — the full graph, interrupt and resume included, runs in pytest without an API key, exactly like `test_pipeline.py` does today. FTS5 store tested against a tiny committed corpus.
- **Golden fixture**: the §12 scenario asserted end-to-end (flags fired, concern present with evidence, veto honored, FK graph complete).
- **Eval set (the real asset)**: 50–200 physician-authored reference plans with rubrics over hdh's synthetic charts — authored in-house (the maintainer is a physician), becoming the module's HealthBench-style spec: acceptance gate for prompt changes, regression suite for model upgrades.

- **Never:** fine-tuning for clinical knowledge. Retrieval provides freshness, provenance, and revocability that weights cannot; the eval set plus the deterministic validators are the reward function if RL-style optimization ever becomes interesting.

14. Phased implementation plan

Phase	Delivers	Proves
1	Schema module (entities + registry specs), FTS5 knowledge store, corpus format + <code>ingest</code> , med-safety rules	data model, registry new-entity path, offline retrieval
2	Subagent graph nodes 1–7 with fakes-first tests; CLI <code>generate/show</code>	constrained generation, section tools, traceability validation
3	Auto-evaluation (rubric corpus + grader + revise loop)	rubric-driven quality gate
4	Checkpoint + interrupt + <code>review/resume/approve/edit/reject</code>	durable human-in-the-loop
5	FHIR export + <code>/CarePlan</code> endpoint; agent-pipeline tool registration; vector-store extra; eval-set harness	end-to-end integration

Each phase lands green through the full gate chain (`just qa`) and ships its guide section; phases 1–2 are the minimum demonstrable slice.

15. Open questions

1. **Embedding provider for the vector store** — local model (no API, heavier install) vs API embeddings (simple, costs, another key)? Default remains FTS5 either way; likely resolve in Phase 5.
2. **Corpus depth vs breadth for v1** — propose: deep on T2DM-elderly + SDOH (the fixture) rather than shallow across many conditions.
3. **Seizure-action-plan-style sub-documents** (distinct structured artifacts attached to a plan) — out of scope for v1, note as the epilepsy scenario's distinctive requirement.
4. **Multi-author collaboration** (patient/caregiver proposals as `intent=proposal` rows) — v2; the `source` field and status model leave room for it.
5. **PlanDefinition/\$apply fidelity** — our condition→template library is a simplified analog; do we eventually encode applicability rules in a CQL-like declarative form, or is that beyond educational scope?

16. References

- Reference analysis: *What a care plan actually is, structurally* (author's design notes, not distributed with the repo)
- HL7 MCC eCare Plan IG (STU1); Gravity Project SDOH Clinical Care IG; Personal Functioning and Engagement IG; CPG-on-FHIR
- C-CDA R2.1 Care Plan document; nursing process (ADPIE)

- hdh internals this design builds on: `docs/ARCHITECTURE.md` (pipeline, schema registry, tracing), `src/hdh/modules/agent/pipeline/`, `src/hdh/core/schema_registry.py`